

Ajmal_Assignment6

December 1, 2025

Part 1- Digit generator

```
[1]: import glob
import imageio
import matplotlib.pyplot as plt
import numpy as np
import os
import PIL
from tensorflow.keras import layers
import time

from IPython import display
```

```
[3]: #a- Reading data
import tensorflow as tf
(train_images, train_labels), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).
    ↳astype('float32')
train_images = (train_images - 127.5) / 127.5 # Normalize the images to [-1, 1]
BUFFER_SIZE = 60000
BATCH_SIZE = 256
# Batch and shuffle the data
train_dataset = tf.data.Dataset.from_tensor_slices(train_images).
    ↳shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 0s

0us/step

```
[4]: # Creating the generative model function
def make_generator_model():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))
```

```

    assert model.output_shape == (None, 7, 7, 256) # Note: None is the batch
    ↪size

    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),
    ↪padding='same', use_bias=False))
    assert model.output_shape == (None, 7, 7, 128)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),
    ↪padding='same', use_bias=False))
    assert model.output_shape == (None, 14, 14, 64)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same',
    ↪use_bias=False, activation='tanh'))
    assert model.output_shape == (None, 28, 28, 1)

    return model

```

```

[5]: # Use the generator model which is not trained yet to create an image
generator = make_generator_model()

noise = tf.random.normal([1, 100]) #This creates the noise
generated_image = generator(noise, training=False)

plt.imshow(generated_image[0, :, :, 0], cmap='gray')

```

```

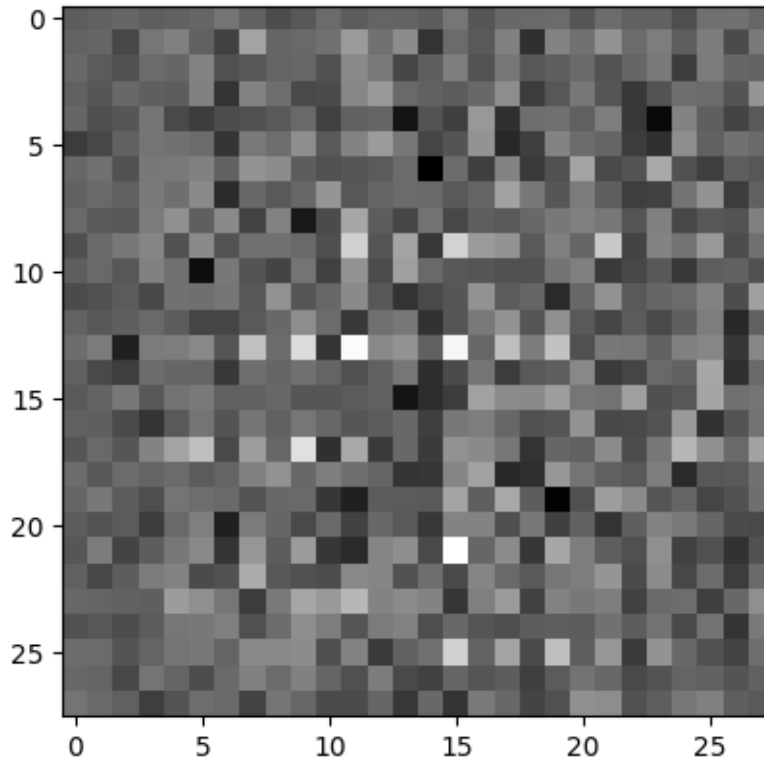
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

[5]: <matplotlib.image.AxesImage at 0x7892d6101940>

```



```
[ ]: # what is this image?

# This image is random noise generated by an untrained generator model.
# We are seeing meaningless patterns because the generator has not learned to
  ↳ create recognizable digits yet.
# The output appears as random grayscale noise since the model's weights are
  ↳ randomly initialized and have not been optimized through training.
```

```
[6]: #c- Create discriminator model
def make_discriminator_model():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same',
                             input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1))
```

```
return model
```

```
[7]: # Applying discriminative model on the untrained image which was the output of
      ↪generative model
discriminator = make_discriminator_model()
decision = discriminator(generated_image)
print(decision)
```

```
/usr/local/lib/python3.12/dist-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
tf.Tensor([[0.00065932]], shape=(1, 1), dtype=float32)
```

```
[8]: # This method returns a helper function to compute cross entropy loss
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
cross_entropy
```

```
[8]: <LossFunctionWrapper(<function binary_crossentropy at 0x7892fc9a3b00>,
kwargs={'from_logits': True, 'label_smoothing': 0.0, 'axis': -1})>
```

```
[9]: #d- Compute the loss
# Discriminator loss This method quantifies how well the discriminator is able
      ↪to distinguish real images from fakes.
# It compares the discriminator's predictions on real images to an array of 1s,
      ↪and the discriminator's predictions on fake (generated) images to an array
      ↪of 0s.
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss
```

```
[10]: #Generator loss The generator's loss quantifies how well it was able to trick
      ↪the discriminator.
#Intuitively, if the generator is performing well, the discriminator will
      ↪classify the fake images as real (or 1). Here, we will compare the
      ↪discriminators decisions on the generated images to an array of 1s.
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)
```

```
[11]: #The discriminator and the generator optimizers are different since we will
      ↪train two networks separately.
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

```
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

```
[12]: # e- Model design
# Let's save the model for future references
checkpoint_dir = '/content/training_checkpoints'
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(generator_optimizer=generator_optimizer,
                                  ↵
                                  ↪discriminator_optimizer=discriminator_optimizer,
                                  generator=generator,
                                  discriminator=discriminator)
```

```
[13]: # Model parameters
EPOCHS = 50
noise_dim = 100
num_examples_to_generate = 16

# We will reuse this seed overtime (so it's easier)
# to visualize progress in the animated GIF)
seed = tf.random.normal([num_examples_to_generate, noise_dim])
```

```
[14]: #The training loop begins with generator receiving a random seed as input. That ↵
      ↪seed is used to produce an image.
#The discriminator is then used to classify real images (drawn from the ↵
      ↪training set) and fakes images (produced by the generator).
#The loss is calculated for each of these models, and the gradients are used to ↵
      ↪update the generator and discriminator.
# Notice the use of `tf.function`
# This annotation causes the function to be "compiled".
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

        gradients_of_generator = gen_tape.gradient(gen_loss, generator.
        ↪trainable_variables)
        gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.
        ↪trainable_variables)
```

```

generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.
↳ trainable_variables))
discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
↳ discriminator.trainable_variables))

```

```

[15]: def train(dataset, epochs):
    for epoch in range(epochs):
        start = time.time()

        for image_batch in dataset:
            train_step(image_batch)

        # Produce images for the GIF as we go
        display.clear_output(wait=True)
        generate_and_save_images(generator,
                                epoch + 1,
                                seed)

        # Save the model every 15 epochs
        if (epoch + 1) % 15 == 0:
            checkpoint.save(file_prefix = checkpoint_prefix)

        print ('Time for epoch {} is {} sec'.format(epoch + 1, time.time()-start))

    # Generate after the final epoch
    display.clear_output(wait=True)
    generate_and_save_images(generator,
                            epochs,
                            seed)

```

```

[16]: def generate_and_save_images(model, epoch, test_input):
    # Notice `training` is set to False.
    # This is so all layers run in inference mode (batchnorm).
    predictions = model(test_input, training=False)

    fig = plt.figure(figsize=(4,4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i+1)
        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
        plt.axis('off')

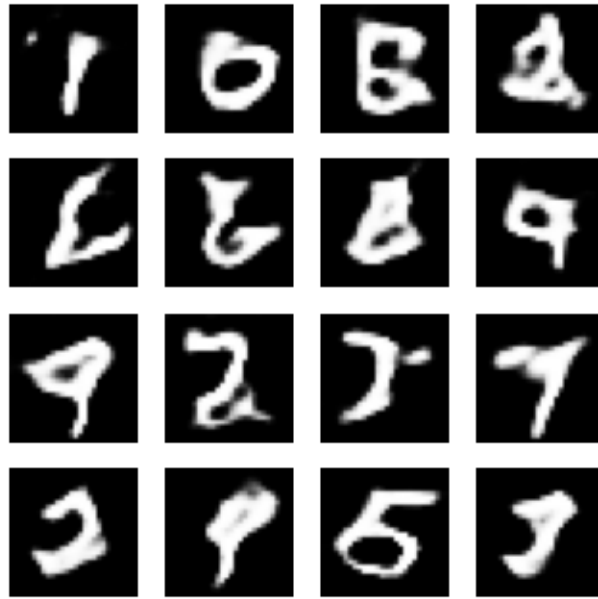
    plt.savefig('image_at_epoch_{:04d}.png'.format(epoch))
    plt.show()

```

```

[17]: #Call the function in proper way
train(train_dataset, EPOCHS) #Put appropriate values here and call the function

```



```
[18]: checkpoint.restore(tf.train.latest_checkpoint(checkpoint_dir))
```

```
[18]: <tensorflow.python.checkpoint.checkpoint.CheckpointLoadStatus at 0x7892402af770>
```

```
[66]: # You can use these commands to save the evolution as a GIF file
anim_file = 'GAN_digit.gif'
```

```
with imageio.get_writer(anim_file, mode='I') as writer:
    filenames = glob.glob('image*.png')
    filenames = sorted(filenames)
    last = -1
    for i,filename in enumerate(filenames):
        frame = 2*(i**0.5)
        if round(frame) > round(last):
            last = frame
        else:
            continue
        image = imageio.imread(filename)
        writer.append_data(image)
        image = imageio.imread(filename)
        writer.append_data(image)

from IPython.display import Image
Image(filename=anim_file)
```

/tmp/ipython-input-3083988527.py:14: DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to that of iio.v3.imread. To keep

the current behavior (and make this warning disappear) use ``import imageio.v2 as imageio`` or call ``imageio.v2.imread`` directly.

```
image = imageio.imread(filename)
/tmp/ipython-input-3083988527.py:16: DeprecationWarning: Starting with ImageIO
v3 the behavior of this function will switch to that of iio.v3.imread. To keep
the current behavior (and make this warning disappear) use `import imageio.v2 as
imageio` or call `imageio.v2.imread` directly.
image = imageio.imread(filename)
```

[66]: <IPython.core.display.Image object>

```
[ ]: # what is your understanding from this GIF file?

# We can observe the evolution of the generator's performance over 50 epochs.
# At the beginning, the generated images appear as random noise with no
↳recognizable patterns.
# As training progresses through the epochs, the generator gradually learns to
↳create shapes that increasingly resemble handwritten digits.
# By the final epochs, we can see that the generator has learned to produce
↳digit-like structures, though they may still be somewhat blurry.
# This demonstrates how the adversarial training process between the generator
↳and discriminator improves the quality of generated images over time.
```

Part 2- Pokemon characters and GAN

```
[ ]: #Install d2l package
!pip install d2l==1.0.0-alpha1.post0
```

```
[22]: import tensorflow as tf
from d2l import tensorflow as d2l
```

```
[23]: #a- Read the dataset and store it
d2l.DATA_HUB['pokemon'] = (d2l.DATA_URL + 'pokemon.zip',
                           'c065c0e2593b8b161a2d7873e42418bf6a21106c')

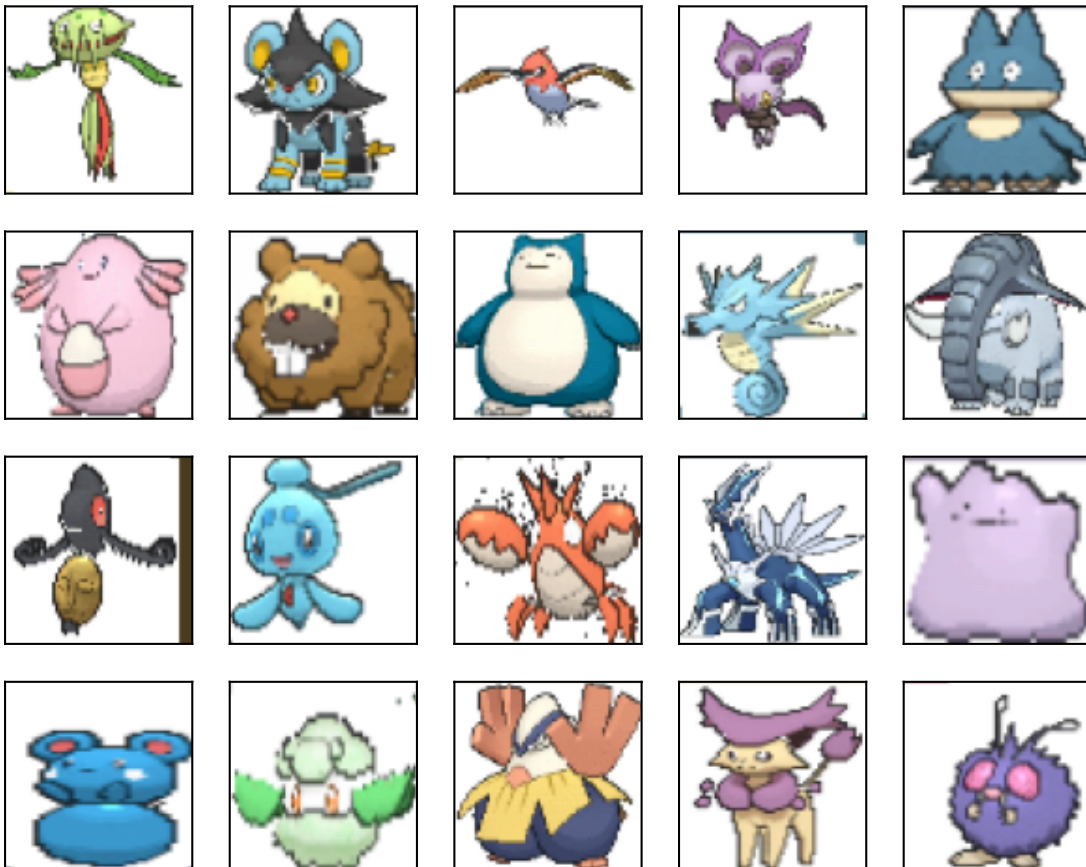
# We store the directory in this variable
data_dir = d2l.download_extract('pokemon')
batch_size = 256
# Use preprocessing from Keras to read images from directory using given batch
↳size and map them to 64x64 images
pokemon = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir, batch_size=batch_size, image_size=(64, 64))
```

Downloading ../data/pokemon.zip from
http://d2l-data.s3-accelerate.amazonaws.com/pokemon.zip...
Found 40597 files belonging to 721 classes.


```
[25]: #Normalize the data between [-1,1]
# We normalize the data with 0.5 mean and 0.5 standard deviation to match
# the value range
# Write a function to normalize the images by dividing them to 255 and use
# gaussian normalization with mean of 0.5 and standard deviation of 0.5.
def transform_func(X):
    # Write the function here in which it normalized the X in the way explained
    # above.
    return (X / 255.0 - 0.5) / 0.5

data_iter = pokemon.map(lambda x, y: (transform_func(x), y),
                        num_parallel_calls=tf.data.experimental.AUTOTUNE)
data_iter = data_iter.cache().shuffle(buffer_size=1000).prefetch(
    buffer_size=tf.data.experimental.AUTOTUNE)
```

```
[26]: #b- Visualize first 20 images
d2l.set_figsize(figsize=(4, 4))
for X, y in data_iter.take(1):
    imgs = X[:20, :, :, :] / 2 + 0.5
    d2l.show_images(imgs, num_rows=4, num_cols=5)
```



0.1 The Generator

The generator needs to map the noise variable to a RGB image with width and height to be 64×64 .

```
[27]: #c- Creating the generator
class G_block(tf.keras.layers.Layer):
    def __init__(self, out_channels, kernel_size=4, strides=2, padding="same",
                  **kwargs):
        super().__init__(**kwargs)
        self.conv2d_trans = tf.keras.layers.Conv2DTranspose(
            out_channels, kernel_size, strides, padding, use_bias=False)
        self.batch_norm = tf.keras.layers.BatchNormalization()
        self.activation = tf.keras.layers.ReLU()

    def call(self, X):
        return self.activation(self.batch_norm(self.conv2d_trans(X)))
```

```
[28]: # Explain your understanding of the G_block here.

# We understand that the G_block (Generator block) is a building block used to
↳construct the generator network.
# It uses Conv2DTranspose layers (also called deconvolutional or upsampling
↳layers) to gradually increase the spatial dimensions of the input.
# Each G_block typically consists of a transposed convolution layer followed by
↳batch normalization and an activation function (like ReLU).
# By stacking multiple G_blocks together, we can progressively upsample a small
↳noise vector into a larger image.
# In this case, the block helps transform the initial noise into a 64x64 RGB
↳Pokemon image by increasing the spatial resolution while learning the
↳features that characterize Pokemon characters.
```

```
[29]: x = tf.zeros((2, 16, 16, 3)) # Input based on the instruction
#Call the G_block package and use 20 as the number of layers (input of the
↳class)
g_blk = G_block(20)
# Show the dimension of the output
g_blk(x).shape
```

```
[29]: TensorShape([2, 32, 32, 20])
```

The generator consists of four basic blocks that increase input's both width and height from 1 to 32. At the same time, it first projects the latent variable into 64×8 channels, and then halve the channels each time. At last, a transposed convolution layer is used to generate the output. It further doubles the width and height to match the desired 64×64 shape, and reduces the channel

size to 3. The tanh activation function is applied to project output values into the $(-1, 1)$ range.

```
[30]: n_G = 64
net_G = tf.keras.Sequential([
    # Output: (4, 4, 64 * 8)
    G_block(out_channels=n_G*8, strides=1, padding="valid"),
    G_block(out_channels=n_G*4), # Output: (8, 8, 64 * 4)
    G_block(out_channels=n_G*2), # Output: (16, 16, 64 * 2)
    G_block(out_channels=n_G), # Output: (32, 32, 64)
    # Output: (64, 64, 3)
    tf.keras.layers.Conv2DTranspose(
        3, kernel_size=4, strides=2, padding="same", use_bias=False,
        activation="tanh")
])
```

```
[31]: # Generate a 100 dimensional latent variable to verify the generator's output
      ↪ shape.
x = tf.zeros((1, 1, 1, 100))
net_G(x).shape
```

```
[31]: TensorShape([1, 64, 64, 3])
```

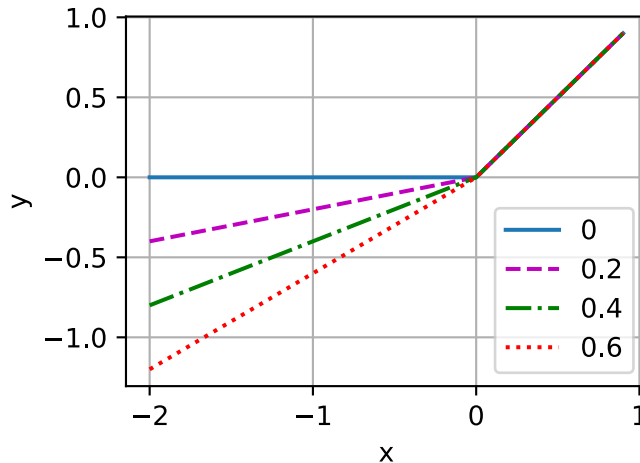
0.2 Discriminator

The discriminator is a normal convolutional network network except that it uses a leaky ReLU as its activation function. Given $\alpha \in [0, 1]$, its definition is

$$\text{leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}.$$

As it can be seen, it is normal ReLU if $\alpha = 0$, and an identity function if $\alpha = 1$. For $\alpha \in (0, 1)$, leaky ReLU is a nonlinear function that give a non-zero output for a negative input. It aims to fix the “dying ReLU” problem that a neuron might always output a negative value and therefore cannot make any progress since the gradient of ReLU is 0.

```
[32]: alphas = [0, .2, .4, .6, .8, 1]
x = tf.range(-2, 1, 0.1)
Y = [tf.keras.layers.LeakyReLU(alpha)(x).numpy() for alpha in alphas]
d2l.plot(x.numpy(), Y, 'x', 'y', alphas)
```



The basic block of the discriminator is a convolution layer followed by a batch normalization layer and a leaky ReLU activation. The hyperparameters of the convolution layer are similar to the transpose convolution layer in the generator block

```
[33]: # d- Block of Discriminator
class D_block(tf.keras.layers.Layer):
    def __init__(self, out_channels, kernel_size=4, strides=2, padding="same",
                  alpha=0.2, **kwargs):
        super().__init__(**kwargs)
        self.conv2d = tf.keras.layers.Conv2D(out_channels, kernel_size,
                                              strides, padding, use_bias=False)
        self.batch_norm = tf.keras.layers.BatchNormalization()
        self.activation = tf.keras.layers.LeakyReLU(alpha)

    def call(self, X):
        return self.activation(self.batch_norm(self.conv2d(X)))
```

```
[46]: # Explain your understanding of D_block here.
# We understand that the D_block (Discriminator block) is a building block used
# to construct the discriminator network.
# It typically consists of a series of convolutional layers followed by batch
# normalization and activation functions (like Leaky ReLU).
# The discriminator's role is to evaluate whether a given input (real or
# generated) is authentic or fake.
# By stacking multiple D_blocks together, we can create a deep discriminator
# network that can effectively distinguish between real and generated data.
```

```
[34]: x = tf.zeros((2, 16, 16, 3))
#Call the G_block package and use 20 as the number of layers (input of the
#class)
```

```
d_blk = D_block(20)
# Show the dimension of the output
d_blk(x).shape
```

[34]: TensorShape([2, 8, 8, 20])

The discriminator is a mirror of the generator.

```
[35]: n_D = 64
net_D = tf.keras.Sequential([
    D_block(n_D), # Output: (32, 32, 64)
    D_block(out_channels=n_D*2), # Output: (16, 16, 64 * 2)
    D_block(out_channels=n_D*4), # Output: (8, 8, 64 * 4)
    D_block(out_channels=n_D*8), # Output: (4, 4, 64 * 64)
    # Output: (1, 1, 1)
    tf.keras.layers.Conv2D(1, kernel_size=4, use_bias=False)
])
```

```
[36]: #It uses a convolution layer with output channel 1 as the last layer to
      ↪ obtain a single prediction value.
x = tf.zeros((1, 64, 64, 3))
net_D(x).shape
```

[36]: TensorShape([1, 1, 1, 1])

0.3 Training

```
[55]: def train(net_D, net_G, data_iter, num_epochs, lr, latent_dim,
              device=d2l.try_gpu()):
    loss = tf.keras.losses.BinaryCrossentropy(
        from_logits=True, reduction=tf.keras.losses.Reduction.SUM)

    for w in net_D.trainable_variables:
        w.assign(tf.random.normal(mean=0, stddev=0.02, shape=w.shape))
    for w in net_G.trainable_variables:
        w.assign(tf.random.normal(mean=0, stddev=0.02, shape=w.shape))

    optimizer_hp = {"learning_rate": lr, "beta_1": 0.5, "beta_2": 0.999}
    optimizer_D = tf.keras.optimizers.Adam(**optimizer_hp)
    optimizer_G = tf.keras.optimizers.Adam(**optimizer_hp)

    animator = d2l.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[1, num_epochs], nrow=2, figsize=(5, 5),
                           legend=['discriminator', 'generator'])
    animator.fig.subplots_adjust(hspace=0.3)

    for epoch in range(1, num_epochs + 1):
        # Train one epoch
```

```

timer = d2l.Timer()
metric = d2l.Accumulator(3) # loss_D, loss_G, num_examples
for X, _ in data_iter:
    batch_size = X.shape[0]
    Z = tf.random.normal(mean=0, stddev=1,
                          shape=(batch_size, 1, 1, latent_dim))
    metric.add(d2l.update_D(X, Z, net_D, net_G, loss, optimizer_D),
               d2l.update_G(Z, net_D, net_G, loss, optimizer_G),
               batch_size)

    # Show generated examples
    Z = tf.random.normal(mean=0, stddev=1, shape=(21, 1, 1, latent_dim))
    # Normalize the synthetic data to N(0, 1)
    fake_x = net_G(Z) / 2 + 0.5
    imgs = tf.concat([tf.concat([fake_x[i * 7 + j] for j in range(7)],
                                axis=1)
                      for i in range(len(fake_x) // 7)], axis=0)
    animator.axes[1].cla()
    animator.axes[1].imshow(imgs)
    # Show the losses
    loss_D, loss_G = metric[0] / metric[2], metric[1] / metric[2]
    animator.add(epoch, (loss_D, loss_G))
    print(f'loss_D {loss_D:.3f}, loss_G {loss_G:.3f}, '
          f'{metric[2] / timer.stop():.1f} examples/sec on {str(device.
↵_device_name)}')

```

[42]: # Explain your understanding from this model here?

```

# We understand that this model is a Generative Adversarial Network (GAN) for
↵generating Pokemon images.
# It consists of two main components: the generator and the discriminator.
# The generator takes a random noise vector as input and attempts to create
↵realistic Pokemon images.
# The discriminator, on the other hand, evaluates the authenticity of the
↵generated images by distinguishing between real Pokemon images and those
↵created by the generator.
# The model is trained to improve the generator's ability to create convincing
↵Pokemon images while the discriminator becomes better at distinguishing
↵between real and generated images.
# This adversarial training process helps the generator learn to create
↵increasingly realistic Pokemon images over time.

```

```

[ ]: # Creating the generator network (6 layers to reach 64x64 from 1x1)
n_G = 64
net_G = tf.keras.Sequential([
    G_block(n_G*16, strides=2, padding='same', input_shape=(1,1,latent_dim)),
    G_block(n_G*8, strides=2, padding='same'),

```

```

        G_block(n_G*4, strides=2, padding='same'),
        G_block(n_G*2, strides=2, padding='same'),
        G_block(n_G, strides=2, padding='same'),
        tf.keras.layers.Conv2DTranspose(3, kernel_size=4, strides=2,
padding='same', use_bias=False, activation='tanh')
    ])

```

/tmp/ipython-input-2459697666.py:5: UserWarning: Do not pass an
`input\_shape`/`input\_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

[63]: *# Creating the discriminator network (4 blocks to downsample 64x64 to 4x4)*

```

n_D = 64
net_D = tf.keras.Sequential([
    D_block(n_D, input_shape=(64,64,3)),
    D_block(n_D*2),
    D_block(n_D*4),
    D_block(n_D*8),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1)
])

```

/tmp/ipython-input-3277077847.py:5: UserWarning: Do not pass an
`input\_shape`/`input\_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

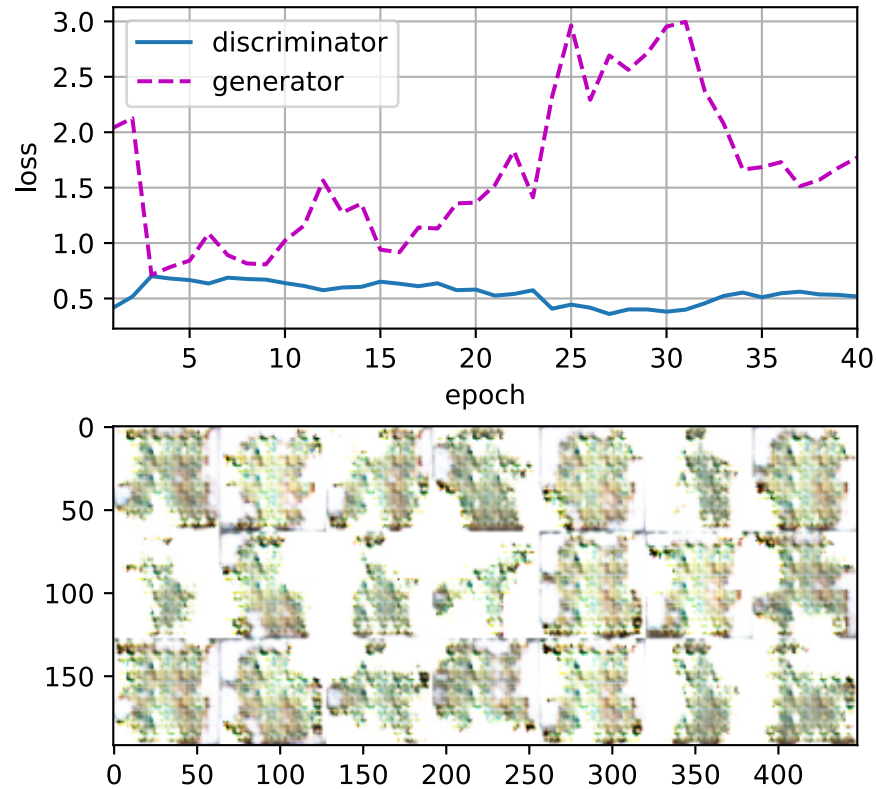
[64]: *# Train the model*

```

latent_dim, lr, num_epochs = 100, 0.0005, 40
train(net_D, net_G, data_iter, num_epochs, lr, latent_dim)

```

loss_D 0.518, loss_G 1.773, 502.9 examples/sec on /GPU:0



0.4 What is your understanding from this learning curve?

1. **Oscillations are Normal:** Unlike standard classification models where loss decreases smoothly, GAN loss graphs typically show significant fluctuations and oscillations. This is expected because the Generator and Discriminator are playing a “zero-sum game”, when one gets better, the other’s loss typically increases.
2. **Discriminator vs. Generator:**
 - **Discriminator Loss:** Ideally, this shouldn’t drop to zero. If it does, it means the discriminator is too strong and the generator stops learning (vanishing gradients). It usually stabilizes around a value (often near 0.5 to 0.7) if the training is balanced.
 - **Generator Loss:** This indicates how well the generator is fooling the discriminator. We want this to decrease or stabilize, but it often spikes when the discriminator learns a new feature that distinguishes real from fake.
3. **Convergence:** In a successful GAN training, we don’t necessarily look for the loss to reach 0. Instead, we look for a **Nash Equilibrium** where both models are strong, and the losses stabilize or oscillate around a constant value, indicating that the generator is producing images that are indistinguishable from real data, and the discriminator is guessing with ~50% accuracy.
4. **Visual Confirmation:** The loss graph alone can be misleading in GANs. The true measure

of performance is the visual quality of the generated Pokemon images over the epochs.